Jupyter to PDFr .sh

Prompt: `jupyter_pdfr.sh`

One-line summary

`jupyter_pdfr.sh` is an interactive, menu-driven Bash utility that batch-converts Jupyter notebooks (`.ipynb`) into HTML or PDF, with optional code-hiding, optional multi-file stitching into a single master document, optional output-folder isolation, and built-in overwrite protection.

Purpose and audience

It's a wrapper around `jupyter nbconvert` (plus `pdftk`/`pdfunite` for PDF stitching and `xelatex` for PDF rendering) intended for a **non-technical or impatient user** who just wants to point at a folder of notebooks and get clean, shareable HTML or PDF out the other end — without remembering nbconvert's flags, without hand-writing a `find | xargs` line, and without ever clobbering an existing file. The whole thing runs from a numbered prompt menu; the user never has to type a raw command.

**Let’s start with what it is *not* **

- It does **not** recurse into sub-directories of the Source Dir (`-maxdepth 1`).
- It does **not** modify, move, or delete the original `.ipynb` files.
- It does **not** support formats other than HTML and PDF (though the `FORMAT` variable is the only thing you'd change to add one).
- It does **not** preserve the notebook's internal per-cell ordering when stitching HTML beyond "the notebooks appear in the order you selected, each body inlined whole."
- It has **no** CLI mode — every decision is interactive. (Making it scriptable would mean factoring the prompt answers into flags or env vars.)

Interface contract

Inputs (all via interactive prompts, no CLI flags):

- Source directory, output directory
- Format (HTML or PDF)
- Code visibility (with / without)
- Stitch? → master name → keep individuals?
- New sub-folder? → folder name
- Notebook selection → all, or ordered indices

Outputs:

- One `<name>.<html|pdf>` per selected notebook, *and/or*
- One master stitched file, *and optionally* both
- All landing in either the Base Output Dir or a named sub-folder
- Exit code `0` on clean quit, `1` on fatal dependency failure

Side effects:

- `sudo apt-get install …` if the user consents at the dependency gate
- Temp files under `/tmp/pdfr_stage_<pid>/`, always cleaned up
- Created sub-folders under the Base Output Dir (only if the user asked for one)

High-level flow

check_deps → verify jupyter / nbconvert / xelatex / pdftk|pdfunite exist

init_directories → ask for Source Dir and Base Output Dir (Enter = cwd)

┌─ main menu loop ──┐

│ 1) HTML 2) PDF 3) Change Directory q) Quit │

└──┘

↓ (if 1 or 2\)

configure_pipeline → linear questionnaire:

A. Include code cells? (Standard / Clean)

B. Stitch into master? → name? → keep individuals?

C. Output to new sub-folder? → name?

D. Pick notebooks: all, or custom ordered subset

↓

execute_conversion → convert each notebook into /tmp sandbox,

move results (collision-safe) into FINAL_OUT_DIR,

stitch if requested, clean up staging

↓

back to main menu

Key behaviors, in detail

1. Startup — dependency gate

On launch, the script checks for `jupyter`, `jupyter nbconvert`, `xelatex`, and either `pdftk` or `pdfunite`. If anything is missing it lists what's absent and offers a one-shot `sudo apt-get install` of the usual Debian/Ubuntu package set (`jupyter-core`, `jupyter-nbconvert`, `pdftk`, `poppler-utils`, `texlive-xetex`, `pandoc`). The user can decline and the script exits cleanly.

2. Directory initialization

Prompts for a **Source Directory** and a **Base Output Location**. Both accept a bare Enter to fall back to the current working directory (so a double-Enter gets you running immediately). Paths are canonicalized with `realpath`; if the source doesn't exist or the output can't be created, it loops back and re-asks rather than aborting — no state is lost.

3. Main menu

A tight four-item menu with an explicit `q. Quit`. Anything unrecognized re-renders the menu. `3. Change Directory` re-invokes the initializer and loops back, so the user can retarget the whole session without restarting.

4. The pre-run questionnaire (only when converting)

A **strictly linear** sequence — each question waits for the previous answer:

- **A. Visibility:** `Standard` (includes code cells) or `Clean` (adds `--no-input`, hiding all code).
- **B. Stitching:** `y` triggers three follow-ups — a *master filename* (extension stripped by the user, sanitized to `[A-Za-z0-9_-]`), then a *keep-individual-files?* choice. `n` skips all of it.
- **C. Folder isolation:** `y` prompts for a new sub-folder name (defaulting to the stitch name if one was given), which is created under the Base Output Dir. `n` dumps straight into the Base Output Dir.
- **D. Selection:** Lists every `.ipynb` found at the top level of the Source Dir, then accepts either *All* or a space-separated, **order-preserving** list of indices (e.g. `3 1 2`). If the user picks a partial subset, it pauses for a confirmation before proceeding.

Every prompt in the whole app is **`q`-abortable** — typing `q` at any question backs out of the current pipeline and returns to the main menu without having written anything.

5. Execution engine

- A per-invocation **transient staging directory** is created under `/tmp` (`/tmp/pdfr_stage_$$`). Every notebook is converted *into* that sandbox first — the user's real output folder is never touched until a conversion has succeeded.
- For each notebook, `jupyter nbconvert --to <fmt> [--no-input]` is run with output visible, so failures are legible.
- On success, the produced file is either:
  - **moved directly** into `FINAL_OUT_DIR` (non-stitch mode), or
  - **queued** in memory for the stitching step (stitch mode).
- **Collision protection:** a `get_safe_path` helper probes the destination and, if a name is taken, appends `(1)`, `(2)`, … until it finds a free slot. Uses `-e`, so it protects against directories too, not just files.
- **Stitching:**
  - PDF → `pdftk … cat output …`, falling back to `pdfunite` if pdftk isn't present.
  - HTML → the script *does not* naively `cat` the files together (which would produce invalid nested `<html>` documents). Instead it extracts each file's `<body>` content and re-wraps everything in a single valid HTML5 document, with CSS giving each notebook its own page-broken, top-bordered `<section>`.
- If *Keep individuals* was chosen during the questionnaire, the staged per-notebook files are moved into `FINAL_OUT_DIR` alongside the master (again through `get_safe_path`).
- The staging area is always removed at the end, success or failure.

6. Output naming

- Non-stitched files keep their original basename, just with the target extension.
- The stitched master takes the name the user supplied (sanitized).
- Existing files are **never overwritten** — duplicates get `name(1).ext`, `name(2).ext`, etc.

Design invariants

Invariant	**How it's enforced**
Never overwrite user data	`get_safe_path` before every `mv`
Never write half-baked output	All conversions happen in `/tmp` and are only moved after success
Never leave orphans	`rm -rf "$staging_area"` at the end of every run
Every prompt is escapable	`prompt_user` returns non-zero on `q`/`Q`, callers `return 1`
No shell-injection via filenames	`printf -v` instead of `eval`; inputs sanitized with `tr -dc`
Accurate visible errors	nbconvert stderr is *not* redirected to `/dev/null`
Portable PDF stitching	Tries `pdftk`, falls back to `pdfunite`
Valid HTML output when stitching	Bodies extracted and re-wrapped, not concatenated